

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное
учреждение высшего образования
Национальный исследовательский Нижегородский государственный университет им. Н.И.
Лобачевского
Институт информационных технологий, математики и механики

Отчет по лабораторной работе

«Реализация метода Гаусса для решения систем линейных уравнений»

Выполнила:

студентка группы 3825Б1ПМии1

Разина В.А.

Проверила:

Бусько П.В.

Нижегород

2026

Содержание

Введение	3
Постановка задачи	4
Руководство пользователя.....	6
Описание программной реализации	7
Результаты экспериментов.....	10
Заключение	10
Литература	11
Приложение	12

Введение

Одна из ключевых задач вычислительной математики — решение систем линейных алгебраических уравнений. Такие системы широко распространены в прикладных областях.

СЛАУ в матричном виде имеет вид: $Ax = b$, где A — это матрица коэффициентов, b — правая часть, x — неизвестные, которые нужно найти.

Метод Гаусса — метод решения СЛАУ, в основе которого лежит исключение неизвестных. Расширенная матрица коэффициентов с помощью последовательных элементарных преобразований строк приводится к более «простому» виду — ступенчатому (верхнетреугольному), в котором легко описать все её решения или понять, что решений нет.

Для программной реализации метода Гаусса матрицу можно рассматривать как вектор векторов, а СЛАУ — матрицу с методом решения системы уравнений. Таким образом, строится иерархия классов, позволяющая использовать механизм наследования в объектно-ориентированном программировании.

Постановка задачи

Разработаем консольное приложение на языке C++, которое будет позволять пользователю с помощью метода Гаусса находить решение системы линейных алгебраических уравнений вида:

$$Ax = b$$

где:

A – квадратная матрица коэффициентов размера $n \times n$;

b – вектор правой части длины n ;

x – искомый вектор неизвестных длины n .

В программе реализуем:

- Шаблонный класс Vector;
- Шаблонный класс SquareMatrix, производный от класса Vector;
- Шаблонный класс SLAU, производный от класса SquareMatrix, который содержит метод Гаусса с выбором ведущего элемента по столбцу.

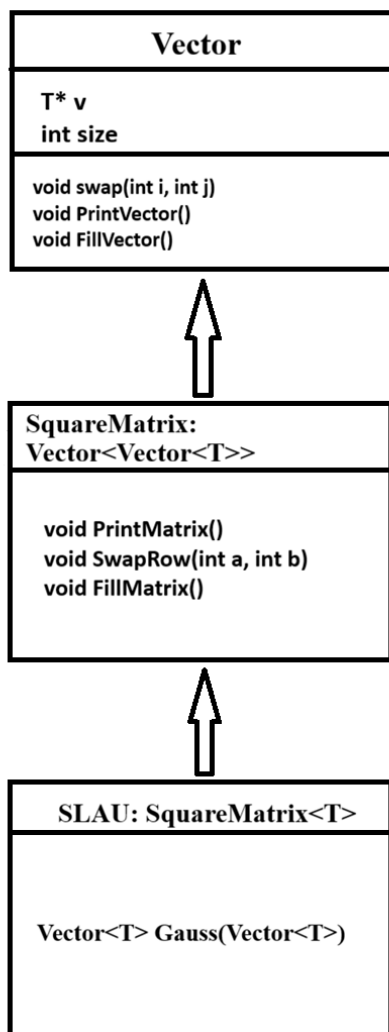


Диаграмма 1. Иерархия классов.

Метод Гаусса будет позволять:

- Обрабатывать системы линейных уравнений с единственным решением;
- Обрабатывать системы линейных уравнений с бесконечным множеством решений и находить частное решение, в котором свободная переменная равна нулю;
- Выявлять несовместные системы, т.е. системы уравнений не имеющие решений;
- Выбирать максимальный по модулю элемент в столбце в качестве ведущего, для повышения точности вычислений.

Руководство пользователя

1. После запуска программы пользователю предлагается ввести размер системы (см. рис 1).

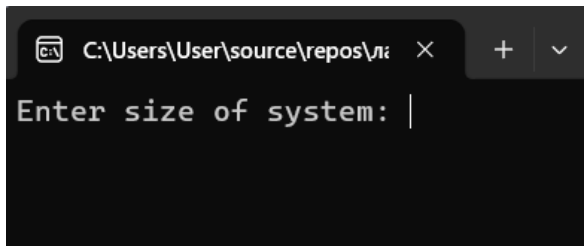


Рис. 1. Ввод размера матрицы.

Далее пользователю будет предложено ввести матрицу коэффициентов. Ввод элементов должен осуществляться через пробел один за другим по строкам. После на консоли выведется матрица, которую ввел пользователь. Далее аналогично производится ввод и вывод правой части системы (см. рис 2).

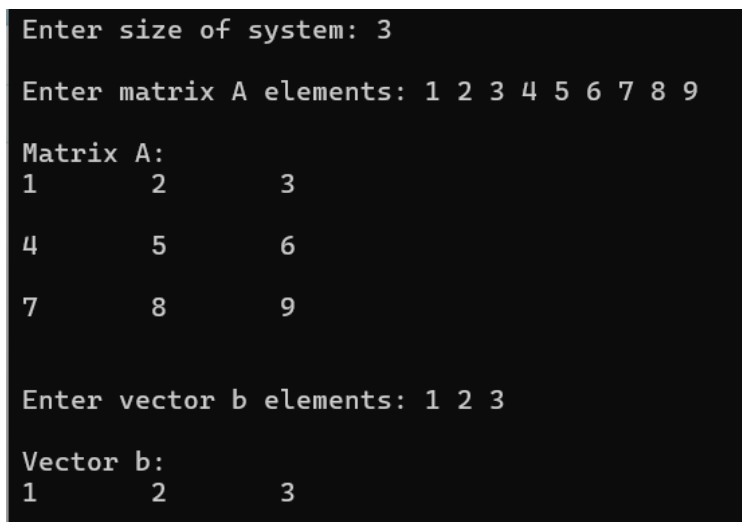


Рис. 2. Ввод системы.

2. После ввода системы программа выводит результат и предлагает ввести заново правую часть или закончить работу с матрицей (см. рис 3).

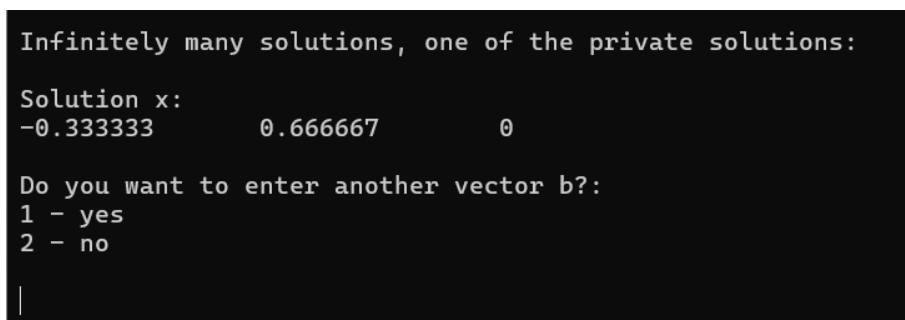


Рис. 3. Пример работы программы.

Описание программной реализации

Программа реализована на языке C++ с использованием объектно-ориентированного подхода, шаблонных классов (позволяют создавать объекты для различных типов данных без дублирования кода) и механизма наследования. Классы, образующие иерархию:

1) `template <typename T> class Vector` – шаблонный класс, предназначенный для хранения и обработки одномерных массивов данных. Содержит поля `int size` (размер вектора) и `T* v` (указатель на массив элементов типа `T`)

Методы класса `Vector`:

- `Vector()` – конструктор по умолчанию, создаёт пустой вектор (нулевой размер, нулевой указатель);
- `Vector(int n, T value = T())` – конструктор с параметрами, выделяет память под `n` элементов и заполняет их значением `value`;
- `Vector(const Vector& copy)` – конструктор копирования;
- `~Vector()` – деструктор, освобождает память;
- `Vector& operator = (const Vector& other)` – перегрузка оператора равно;
- `void swap(int i, int j)` – меняет местами элементы с индексами `i` и `j`;
- `T& operator [] (int i)` – перегрузка доступа по индексу, выполняет проверку границ, при ошибке бросает исключение;
- `const T& operator [] (int i) const` – константный доступ по индексу;
- `void PrintVector()` – выводит элементы вектора;
- `void FillVector()` – заполняет вектор значениями, введёнными с клавиатуры.

2) `template <typename T> class SquareMatrix : public Vector<Vector<T>>` – шаблонный класс, производный от `Vector`. Представляет квадратную матрицу как вектор векторов.

Методы, реализуемые в классе:

- `SquareMatrix(int n) : Vector<Vector<T>>(n, Vector<T>(n, T()))` – конструктор по умолчанию, создаёт квадратную матрицу размером `n × n`, заполненную значениями по умолчанию `T()`;
- `SquareMatrix(int n, T a) : Vector<Vector<T>>(n, Vector<T>(n, a))` – конструктор с параметрами, создаёт квадратную матрицу `n × n`, элементы которой равны `a`;
- `void PrintMatrix()` – построчно выводит матрицу на экран, каждая строка выводится методом `PrintVector()` базового класса;
- `Vector<T>& operator [] (int i)` – перегрузка доступа к строке матрицы по индексу, выполняет проверку границ, возвращает ссылку на `Vector<T>`;
- `const Vector<T>& operator [] (int i) const` – константная перегрузка `[]`;

- `void FillMatrix()` – заполняет матрицу значениями, для каждой строки вызывается `FillVector()`;
- `void SwapRow(int i, int j)` – меняет *i*-ую и *j*-ую строки матрицы местами.

3) `template <typename T> class SLAU : public SquareMatrix<T>` – шаблонный класс, производный от `SquareMatrix<T>`. Представляет систему линейных уравнений и содержит реализацию метода Гаусса.

`SLAU(int n) : SquareMatrix<T>(n)` – конструктор по умолчанию, создаёт систему с матрицей $n \times n$, заполненной нулями;

`SLAU(int n, T a) : SquareMatrix<T>(n, a)` – конструктор с параметрами, создаёт систему с матрицей $n \times n$, все элементы которой равны *a*;

`Vector<T> Gauss(Vector<T> b)` – метод Гаусса для решения системы линейных уравнений, возвращает вектор значений.

В программе реализован метод Гаусса с выбором ведущего элемента, в качестве которого выбирается максимальный по модулю элемент в каждом столбце. Выбор максимального по модулю элемента обеспечивает вычислительную устойчивость алгоритма (предотвращает деление на очень малые числа и минимизирует погрешность округления).

Алгоритм метода Гаусса:

- 1) Создаётся копия матрицы (*copy*) и вектора правой части (*ans*)
- 2) Вектор *pivot* хранит номер ведущего столбца для каждой строки
- 3) Переменная *row* – текущая строка для обработки
- 4) Прямой ход с выбором главного элемента (приводит матрицу к верхнетреугольному виду):
 - для каждого столбца осуществляется поиск строки с максимальным по модулю элементом в текущем столбце
 - если ведущий элемент не равен нулю, то строки меняются местами
 - записывается номер ведущего столбца
 - исключаются элементы под ведущем
 - увеличивается *row*
 - иначе осуществляется переход к следующему столбцу.
- 5) Проверка совместности системы: если после прямого хода остались строки с нулевой левой частью, но соответствующий элемент правой части не ноль — система несовместна.
- 6) Обратный ход
- 7) Для каждой строки, начиная с последней обработанной:

- определяется номер ведущего столбца $c = \text{pivot}[i]$
- вычисляется сумма элементов, расположенных правее
- вычисляется $x[c]$

8) Возвращение результата

Данная реализация метода Гаусса позволяет обрабатывать системы, которые имеют единственное решение или не имеют решения, и системы с бесконечным множеством решений. В последнем случае свободным переменным присваивается значение ноль, а после выводится частное решение для этого случая.

`int main()` – отвечает за пользовательский интерфейс и управляет работой программы:

- Ввод с клавиатуры размера системы;
- Ввод/вывод матрицы коэффициентов;
- Ввод/вывод вектора b ;
- Вызов метода Гаусса;
- Вывод результата или сообщения об ошибке;
- Обработка всех исключений с выводом сообщений.

Результаты экспериментов

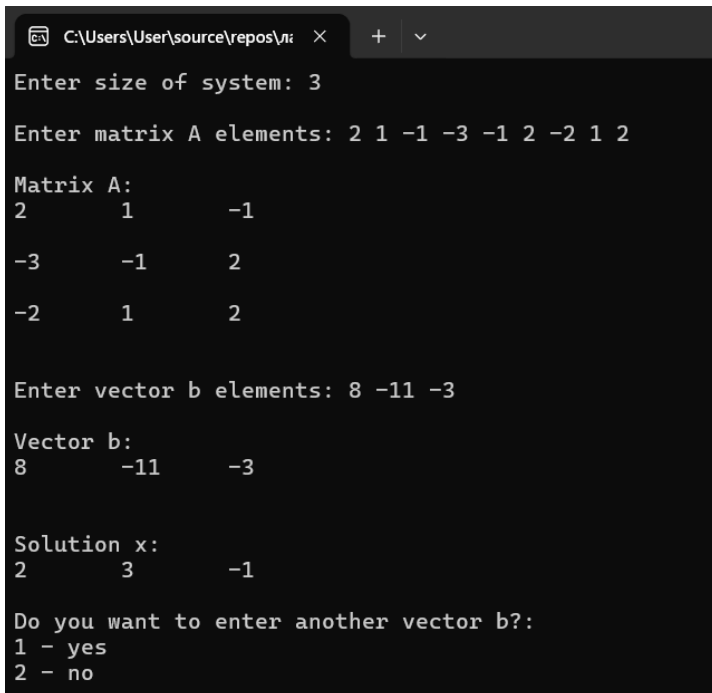
В ходе работы были проведены запуски программы для различных типов систем линейных уравнений: систем с невырожденной матрицей, систем с вырожденной матрицей, систем с малыми коэффициентами, систем с нулевым диагональным элементом, систем с нулевой строкой, систем с плохо обусловленной матрицей.

1) Система с невырожденной матрицей

Рассмотрена система линейных уравнений, матрица коэффициентов которой невырождена (определитель матрицы не равен нулю):

$$\begin{cases} 2x + y - z = 8 \\ -3x - y + 2z = -11 \\ -2x + y + 2z = -3 \end{cases}$$

Система имеет единственное решение (2, 3, -1). В результате работы программы было получено верное решение системы.



```
C:\Users\User\source\repos\...>
Enter size of system: 3
Enter matrix A elements: 2 1 -1 -3 -1 2 -2 1 2
Matrix A:
2      1      -1
-3     -1      2
-2      1      2

Enter vector b elements: 8 -11 -3
Vector b:
8     -11     -3

Solution x:
2      3     -1

Do you want to enter another vector b?:
1 - yes
2 - no
```

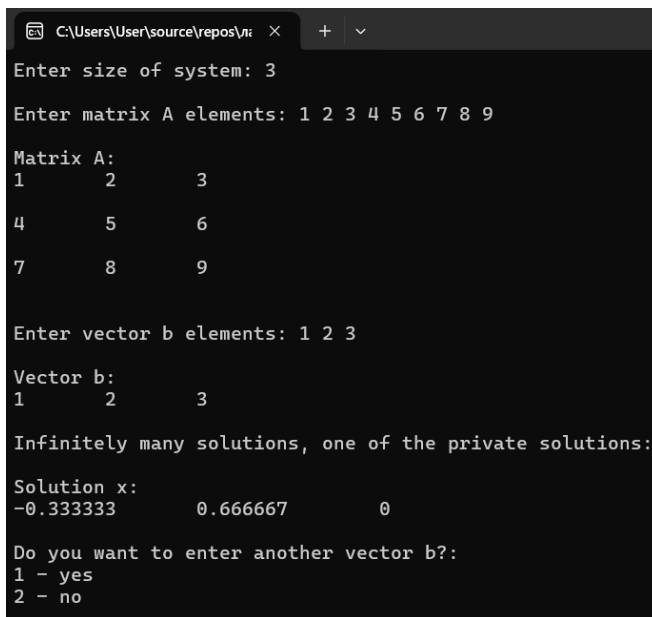
Рис. 4. Работа программы.

2) Система с вырожденной матрицей

Рассмотрена система линейных уравнений, матрица коэффициентов которой имеет определитель, равный нулю:

$$\begin{cases} 1x + 2y + 3z = 1 \\ 4x + 5y + 6z = 2 \\ 7x + 8y + 9z = 3 \end{cases}$$

Данная система имеет бесконечно множество решений $(-1/3+t, 2/3-2t, t)$, где t - любое действительное число. В результате работы программы было получено верное частное решение при $t=0$.



```
C:\Users\User\source\repos\лн >
Enter size of system: 3
Enter matrix A elements: 1 2 3 4 5 6 7 8 9
Matrix A:
1      2      3
4      5      6
7      8      9
Enter vector b elements: 1 2 3
Vector b:
1      2      3
Infinitely many solutions, one of the private solutions:
Solution x:
-0.333333      0.666667      0
Do you want to enter another vector b?:
1 - yes
2 - no
```

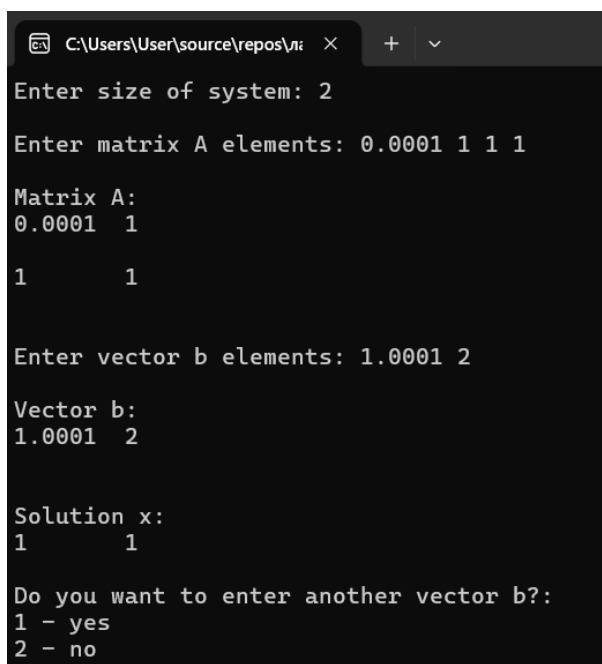
Рис. 5. Работа программы.

3) Система с малыми коэффициентами

Рассмотрена система линейных уравнений:

$$\begin{cases} 0,0001x + y = 1.0001 \\ x + y = 2 \end{cases}$$

В результате работы программы был получен ответ (1, 1), который совпадает с решением системы.



```
C:\Users\User\source\repos\лн >
Enter size of system: 2
Enter matrix A elements: 0.0001 1 1 1
Matrix A:
0.0001  1
1      1
Enter vector b elements: 1.0001 2
Vector b:
1.0001  2
Solution x:
1      1
Do you want to enter another vector b?:
1 - yes
2 - no
```

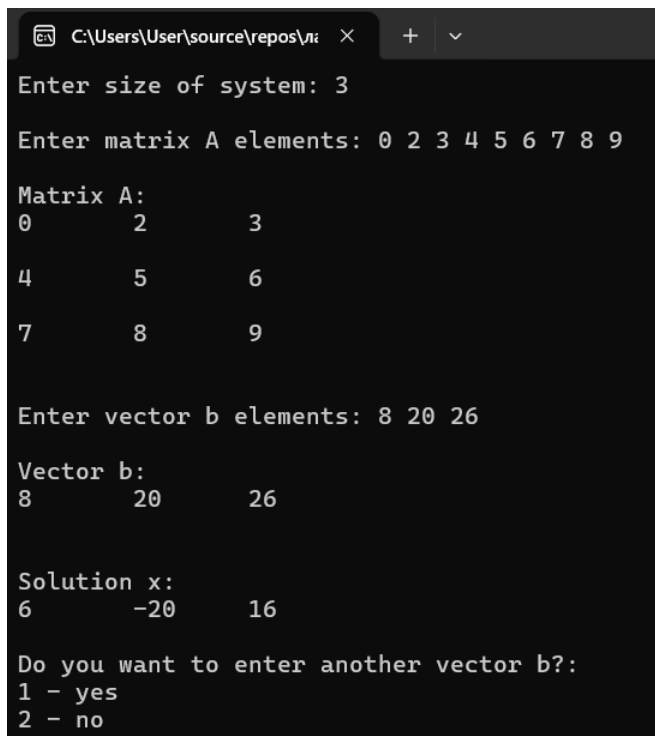
Рис. 6. Работа программы.

4) Система с нулевым диагональным элементом

Рассмотрена система уравнений:

$$\begin{cases} 0x + 2y + 3z = 8 \\ 4x + 5y + 6z = 20 \\ 7x + 8y + 9z = 26 \end{cases}$$

Данная система имеет решение (6, -20, 16). В результате работы программы получен верный ответ.



```
C:\Users\User\source\repos\li > Enter size of system: 3
Enter matrix A elements: 0 2 3 4 5 6 7 8 9
Matrix A:
0      2      3
4      5      6
7      8      9
Enter vector b elements: 8 20 26
Vector b:
8      20      26
Solution x:
6      -20      16
Do you want to enter another vector b?:
1 - yes
2 - no
```

Рис. 7. Работа программы.

5) Систем с нулевой строкой, но ненулевой правой частью

Рассмотрена система линейных уравнений:

$$\begin{cases} 0x + 0y + 0z = 1 \\ 4x + 5y + 6z = 5 \\ 7x + 8y + 9z = 7 \end{cases}$$

Данная система не имеет решений, программа выдала корректный ответ.

```
C:\Users\User\source\repos\лз X + v
Enter size of system: 3
Enter matrix A elements: 0 0 0 4 5 6 7 8 9
Matrix A:
0      0      0
4      5      6
7      8      9

Enter vector b elements: 1 5 7
Vector b:
1      5      7

Error: No solutions

Do you want to enter another vector b?:
1 - yes
2 - no
```

Рис. 8. Работа программы.

Проведенные эксперименты показывают корректность работы программы, из чего следует правильность реализации метода Гаусса с выбором ведущего элемента. Эксперименты с невырожденными матрицами показали, что программа корректно находит решения систем линейных уравнений и получает верные результаты. Исследование вырожденных матриц подтвердило успешность проверки существования решения. Программа верно определяет случаи бесконечного множества решений и приводит одно частное. Эксперимент с малыми коэффициентами показывает важность выбора ведущего элемента: выбор максимального по модулю элемента значительно повышает точность вычислений.

Заключение

В работе был написан код программы, в которой реализован метод Гаусса для решения систем линейных алгебраических уравнений с выбором ведущего элемента. Были написаны шаблонные классы `Vector`, `SquareMatrix`, `SLAU`, реализованы механизмы наследования классов и перегрузки операторов.

Разработанная программа корректно обрабатывает случаи совместных и несовместных систем, а также вырожденных матриц. Использование шаблонов позволяет применять код для различных типов данных (`int`, `float`, `double`).

В ходе вычислительных экспериментов установлено, что метод обеспечивает высокую точность для хорошо обусловленных систем. Метод корректно решает системы различных типов, в том числе системы с малыми коэффициентами.

Литература

1. Керниган Б., Ритчи Д., Фьюэр А. Язык программирования СИ //М.: Финансы и статистика. – 1992.

Приложение